

Qué son?

Cada archivo de código Python es un `módulo`.

Hasta ahora hemos trabajado todo nuestro código en un sólo `módulo`.

Sin embargo, **lo mejor siempre es dividir los proyectos en `módulos` más pequeños.**

Y lo mejor es que estos `módulos` dividan el código *categoricamente*.

Esto lo podemos lograr teniendo el entry point (*punto de entrada*) del código en un `módulo` principal, e importar el resto de las funciones, variables o clases de otros módulos usando la palabra `import` seguida del nombre del `módulo` que queremos importar.

La misma sintaxis que hemos utilizado para importar `librerías` como `csv` o `json` durante las lecciones de Manejo de archivos.

La diferencia es que estas `librerías` están compuestas de muchos `módulos` más pequeños.

Cómo implementarlos

Veamos un ejemplo.

Supongamos que tenemos este código.

```
1 def remove_upper_cases_from_string(input_string):
2     new_string = ""
3     for char in input_string:
4         if not char.isupper():
5             new_string += char
6
7     return new_string
8
9 input_string = "ASI podemos organizar MEJOR nuestro codigo"
10 output_string = remove_upper_cases_from_string(input_string)
11
12 print(output_string)
```

Copiar

podemos organizar nuestro código

Intentemos dividirlo en 2 módulos distintos:

Uno llamado `utilities.py` que tenga la función.

Otro llamado `main.py` que utilice esa función.

```
1 def remove_upper_cases_from_string(input_string):
2     new_string = ""
3     for char in input_string:
4         if not char.isupper():
5             new_string += char
6
7     return new_string
```

Copiar

`utilities.py`

```
1 import utilities
2
3
4 input_string = "ASI podemos organizar MEJOR nuestro codigo"
5 output_string = utilities.remove_upper_cases_from_string(input_string)
6
7 print(output_string)
```

Copiar

main.py



podemos organizar nuestro código

Perfecto! Ahora nuestros módulos son más unitarios y granulares.



Al importar módulos **completos** tenemos que usar su nombre **seguido de un punto** y **la función o variable que queramos acceder**. Por eso, en este caso, es `utilities.remove_upper_cases_from_string`.

En caso de que no necesitemos más cosas de ese `módulo`, podemos importar **solo esa función en específico** usando la palabra `from`.

```
1 from utilities import remove_upper_cases_from_string
2
3
4 input_string = "ASI podemos organizar MEJOR nuestro codigo"
5 output_string = remove_upper_cases_from_string(input_string)
6
7 print(output_string)
```

Copiar

main.py

También podemos importar `n` cantidad de recursos específicos de un módulo separándolos por comas.

```
1 from module import function_a, function_b, function_c, variable
```

Copiar

Otro detalle importante es que al trabajar con módulos, también se suele dividir el proyecto en carpetas con distintos módulos cada una.

Por ejemplo, puedo crear una carpeta llamada `utils` (en caso de tener varios módulos con utilidades de distintos tipos ahí) y renombrar el módulo a `strings`.

```
1 def remove_upper_cases_from_string(input_string):
2     new_string = ""
3     for char in input_string:
4         if not char.isupper():
5             new_string += char
6
7     return new_string
```

Copiar

utils/strings.py

Para importar módulos dentro de otras carpetas, usamos la ruta separada por puntos en vez de slashes /.

En otras palabras, el nombre de la carpeta seguida de un punto y el nombre del módulo.

En este caso, `utils.strings`.

```
1 from utils.strings import remove_upper_cases_from_string
2
3
4 input_string = "ASI podemos organizar MEJOR nuestro codigo"
5 output_string = remove_upper_cases_from_string(input_string)
6
7 print(output_string)
```

Copiar

main.py